Lyapunov exponent estimation for the SRNN: alignment time, Benettin (K = 1), top-K methods, noise, and information rates

2026-09-11

Purpose

This note collects what the numerics-verification work of 2026-09-10/11 established about estimating Lyapunov exponents on this model, what the literature says about the methods, and what should be built next: a top-K Lyapunov spectrum on the full 500-neuron network. It is written so the design of that implementation, and the wording of the manuscript's methods, can be taken from one place.

Companion documents:

- `figs/numerics_verification_trials/numerics_verification_report.md` – the ensemble verification (5 reshoot seeds, 25 LLE seeds per regime).
- `figs/numerics_verification_test_med/interpretation_report_medium.md` – the initial-condition, `lya_dt` and window controls.
- `docs/EquationsParametersDocs/Equations_stability_paper.md` – the model.

Code as of `b4cdb18`: `SRNNCellTypePairs.benettin_algorithm_internal` (K = 1, finite perturbation), `SRNNCellTypePairs.lyapunov_spectrum_qr_internal` (K = N, tangent space), `run_numerics_verification`, `SRNNNumericsProbe`.

1. What a Lyapunov exponent is, operationally

A perturbation to the state, $\delta S(0)$, evolves under the linearised dynamics $\delta S(t) = M(t)\, \delta S(0)$, where M is the fundamental matrix of the variational equation $d\delta S/dt = J(S(t))\, \delta S$ and J is the Jacobian along the fiducial trajectory. Oseledets' theorem says that for almost every $\delta S(0)$ the growth rate $(1/t) \ln |\delta S(t)|$ converges to the largest exponent λ_1, and that the growth rate of the k-volume spanned by k independent perturbations converges to $\lambda_1 + \dots + \lambda_k$. Every practical method is a way of following one or more of these volumes without numerical overflow or collapse.

Two facts govern everything below:

- **Alignment.** A generic vector rotates toward the direction of the largest exponent at rate $(\lambda_1 - \lambda_2)$. Until it has, its growth rate is a weighted average of several exponents.
- **Finite time.** What any run measures is a finite-time exponent over the accumulation window. Its scatter across trajectories is a property of the dynamics, not of the estimator, and it can be large.

2. Alignment time

2.1 The mechanism

Decompose the initial perturbation on the Lyapunov directions: $\delta S(0) = \Sigma\, c_i v_i$. Each component grows or decays as $c_i e^{\lambda_i t}$. The ratio of the second component to the first shrinks as $e^{-(\lambda_1 - \lambda_2) t}$, so the vector is aligned to a factor $1/\rho$ with the leading direction after `t_align = ln(ρ |c_2 / c_1|) / (λ_1 − λ_2).`

Renormalisation (Benettin) or orthonormalisation (QR) rescales the vector; it never rotates it. The rotation comes only from this differential growth. So **alignment time is a property of the spectral gap, not of the method.**

2.2 On this network

The reduced-network QR spectra (`Fig_numerics_lya_method`, row 2) show the gap in each regime:

regime	λ_1	what sits just below λ_1	gap	t_align to 1%
no adaptation (chaotic, full network λ_1 ≈ +3.6)	large and positive	well separated	order 1-10 /s	under a second
single-timescale (intermittent, λ_1 ≈ +0.6)	small positive	a band near zero	~0.5 /s	a few seconds
multiple-timescale (stable, λ_1 ≈ −0.12)	−0.117	a long plateau from −0.12 to −0.5: the 500 neurons' individual slow-adaptation directions	0.01-0.1 /s	**tens of seconds to minutes**

The stable regime is the problem case. Its slowest directions are the individual neurons' 10 s adaptation variables, nearly degenerate, so a random perturbation sheds its faster components only slowly and a 10 s warm-up leaves a fraction of them. The finite-time exponent is then a weighted average of −0.117 and more negative rates. In the ensemble run, Benettin gave −0.117 as the median but scattered ± 0.02 with two seeds at −0.15 and −0.20; QR gave −0.117 ± 0.004 on every seed.

2.3 Why QR did not suffer

Not because a QR vector aligns faster; it does not. Two other reasons:

1. **QR starts from the identity basis.** In the stable regime the slow directions are close to single state coordinates (each neuron's slowest `a` variable), so some basis vectors start already aligned. A random Benettin vector spreads its weight over all 4000 coordinates.
2. **QR reports the best of N.** The exponents are sorted at the end; λ_1 is the largest finite-time rate among all tracked vectors, and at least one started near the slow band. Benettin has one vector.

The flip side, recorded earlier on `SRNNModel2` (see the `lya_warmup` table in that class): when the leading direction is *not* near a coordinate axis, as for the expanding direction of a chaotic network, the identity basis gives QR no head start and it needed more warm-up than Benettin (12% off at 5 s where Benettin was at 0.2%). Neither method is generally faster.

2.4 What to do about it

- In chaotic regimes: nothing; alignment is fast.
- In stable regimes, where λ_1 is wanted to better than ~0.05: warm up for a few times 1/(λ_1 − λ_2) (about 60 s here), or use a K > 1 method (Section 4), which gets the head start from having several vectors.
- The bias, when present, is always toward *more negative* values. It can make a stable network look more stable; it cannot make an unstable one look stable.

3. Benettin's method (K = 1) as implemented

`benettin_algorithm_internal` follows a **finite** perturbation of norm d0 = 1e-3: at each renormalisation time it re-integrates the model from S + δ over `lya_dt` = 0.02 s with the same integrator as the fiducial run, measures the separation, accumulates $\ln(d/d0)/\tau$, and rescales δ back to d0 along the new direction.

Why finite rather than tangent-space. Because both the fiducial and the perturbed trajectory are integrated with the *same* fixed-step scheme on the *same* Brownian path (absolute-time noise indexing in `sde_fixed_step`), the discretisation error and the noise are common to the two and cancel in their difference. That is what makes the LLE measurable with noise on. The ensemble verification confirmed it: paired over 25 seeds, Benettin with SRA1 minus Benettin with ode45 is within 1.4 standard errors of zero in every regime and identical to four decimals in the stable one.

What d0 costs. A finite perturbation samples the nonlinearity at scale d0. With a piecewise activation this could in principle bias the estimate when the two trajectories straddle a breakpoint; the `lya_dt` control (0.01-0.1 s, no change to four digits) and the step-refinement control (no monotone trend) showed no such effect on this network.

Finite-time scatter. Over a 10 s window the exponent scatters across trajectories with sd 0.6 (chaotic), 0.17 (intermittent) and 0.016 (stable), with either integrator; a 1% change of initial condition produces the same scatter. In the chaotic regimes the paired difference between integrators is as wide as the seed-to-seed spread, because after a few Lyapunov times the two integrators are on different trajectories of the same attractor. **Single-realisation exponents in chaotic regimes should be reported as medians over reps**, which the sweeps do.

Warm-up. `lya_warmup` (default 5 s, 10 s in the verification) is iterated before accumulation starts. Adequate in the chaotic regimes; short in the stable one (Section 2).

4. Top-K methods

4.1 The standard algorithm

Benettin et al. (1980) and Shimada & Nagashima (1979): evolve K orthonormal tangent vectors under the variational equation; every τ seconds, orthonormalise them (Gram-Schmidt, or a thin QR of the N × K matrix); the log of the i-th diagonal entry of R, accumulated and divided by time, converges to λ_i. K = 1 is Benettin's LLE in tangent-space form; K = N is the full spectrum; anything between is the partial spectrum. Skokos (2010) gives the algorithm in pseudo-code; Dieci & Van Vleck have a paper specifically on computing a few exponents; Geist, Parlitz & Lauterborn (1990) and Christiansen & Rugh (1997) give continuous orthonormalisation variants (more accurate, not cheaper).

4.2 Cost

Per time step: K Jacobian-vector products plus, at each orthonormalisation, an $O(N K^2)$ thin QR.

	our QR (K = N, `Q = eye(N)`, ode45 on N^2 equations)	top-K, sparse J, fixed step
N = 320 (reduced network)	60-240 s per 20 s run	negligible
N = 4000 (paper's network)	infeasible (16 M variational equations)	K = 10: ~10 × cost of one trajectory; K = 300: affordable, QR ~4e8 flops per orthonormalisation

The Jacobian of this network is sparse (in-degree 100, so ~50 000 non-zeros in the x-block against 16 M entries), and `compute_Jacobian_fast` already returns it sparse. A tangent vector costs one sparse matrix-vector product per stage of the integrator.

Measured (2026-09-12). The first implementation assembled the sparse Jacobian at every step, and that assembly, not the product, was the cost: 15.5 ms per call at N = 4000 (indexed assignment into a sparse matrix) against 0.1-2.6 ms for the product with K = 1-200. `SRNNCellTypePairs.jacobian_times` now applies the Jacobian blocks directly to the N × K basis without forming the matrix, verified equal to the assembled product to 1e-16 (`test_jacobian_times`). Per call at N = 4000:

K	assemble + J·Y	`jacobian_times`	speed-up
1	15.6 ms	0.28 ms	57×
10	16.0 ms	0.64 ms	25×
50	16.7 ms	2.8 ms	6×
200	18.1 ms	11.7 ms	1.6×

At K = 200 the matrix-free routine's dense N × K temporaries (row-block copies in and out of the basis, memory-bound) cost about as much as the assembly it avoids, so the gain flattens; the QR ($O(N\,K^2)$) is still not the limit. On the paper's network (n = 500, 20 s run, 10 s accumulation, noise off; Lyapunov seconds only, trajectory excluded):

regime	N	K = 10	K = 50	K = 200
no adaptation	500	3 → 5	4 → 12	9 → 41
single timescale	2000	29 → 7	32 → 18	44 → 80
multiple timescale	4000	139 → 8	154 → 33	172 → 142

(assembled → matrix-free). Where N is small the assembly was already cheap and one sparse product beats the dense block routine at large K, so the matrix-free path is a win for the full network at K ≤ 50 and a loss for small networks at K = 200. The spectra are identical in every cell (to 1e-13). If K ∼ 200 on the full network ever matters, the next step is to avoid the row-block copies, not to bring the assembly back.

4.3 Why it also fixes the alignment problem

With K vectors, the reported λ_1 is the best-aligned of K after sorting, and the leading directions are recovered even when the single random vector would not have aligned in the warm-up. For the stable regime this is the practical remedy.

4.4 What the top-K spectrum gives beyond λ_1

- **Kolmogorov-Sinai entropy rate.** Pesin's identity: $h\_KS = \Sigma\, \lambda\_i^+$ (nats/s; divide by ln 2 for bits/s), with equality for the natural measure, the standard assumption for dissipative systems of this class (Engelken, Wolf & Abbott 2023 use it for rate networks). **K must reach past the last positive exponent**, and the number of positive exponents is not known in advance: grow K until the smallest tracked exponent is clearly negative and settled. Chaos in rate networks is extensive and the spectrum is symmetric about the mean exponent $-1/\tau$ (Engelken et al.), so in the no-adaptation regime the positive part could be hundreds of exponents at N = 4000; in the intermittent regime a handful; in the stable regime none (h_KS = 0).
- **Kaplan-Yorke dimension.** $D\_KY = k + (\Sigma\_\{i\le k\}\ \lambda\_i)/|\lambda\_\{k+1\}|$ at the k where the cumulative sum crosses zero. Needs K a little beyond the positive exponents. `compute_kaplan_yorke_dimension_internal` already does this and works unchanged on a partial spectrum that contains the crossing.
- **Steadier statistics.** Fluctuations of different exponents are only partly correlated, so the relative finite-time scatter of h_KS is smaller than that of λ_1. Engelken et al. prefer h_KS and D_KY for regime comparisons for this reason, and show both scale linearly with N while λ_1 saturates.

4.5 Extensivity and the reduced network

Engelken et al. (2023) show the Lyapunov spectrum of rate networks is size-invariant when plotted against i/N. That is the assumption behind using a 40-neuron network for the Benettin-vs-QR cross-check, and it held qualitatively (the three regimes appear on the reduced network, and the stable regime's $\lambda_1 = -0.117$ is the same on 40 and 500 neurons). But individual 40-neuron no-adaptation networks ranged from dead fixed points ($\lambda_1 = -10$) to weak chaos (+2), so the reduced network is a check of the *methods*, not a stand-in for the paper's exponents. With top-K on the full network the cross-check becomes a cross-check and nothing more.

5. Stochastic considerations

5.1 The exponents exist and the tangent equation is unchanged

With additive noise on x only, the model is a random dynamical system in Arnold's sense; Oseledets' theorem applies and the exponents are those of the noisy dynamics. Because the noise enters additively, **the Jacobian does not depend on the noise** and the variational equation is exactly the deterministic one along the (stochastic) fiducial trajectory. QR methods for SDEs are in Carbonell, Biscay & Jimenez (2010), following Talay (1991); recent work gives rigorous enclosures for additive-noise flows.

Consequences for our code:

- The QR routine already interpolates a stored fiducial trajectory and integrates the variational system along it; it runs on a noisy SRA1 trajectory as is. The interpolation of a noise-driven x between 2.5 ms samples is slightly rougher than for a smooth trajectory; at this noise level that is a second-order concern.
- Benettin's finite-difference form works with noise *because* both trajectories share one path. A tangent-space K = 1 method works with noise because the tangent equation ignores it. Both are valid; they differ in what they cancel (Section 3).
- The verification ran QR noise-free only. A noisy QR run is a one-flag change and closes the last untested combination.

5.2 Integrator precision under noise

The reshoot experiments showed SRA1's error at 400 Hz is drift-dominated at $\sigma_u = 0.025$ (slopes 1.98-1.82 against a strong-order floor of 1.5), so the integrator behaves the same with noise on as off, and the strong-order term is only glimpsed where the drift error is smallest. The strong order itself (1.73 measured) is verified at large σ in `test_sde_integrators`.

5.3 What noise does to the measured LLE

Nothing systematic: the LLE of the noisy system is a well-defined quantity and Benettin measures it with the noise cancelled in the difference. What noise *does* change is the trajectory-to-trajectory scatter of finite-time exponents, in a direction not yet measured (it could narrow, by mixing the attractor faster, or widen). The reps spread at fixed parameters in the existing sensitivity sweeps is that measurement with noise on.

6. Information rates

6.1 Positive exponents: generation of unpredictability

$h_{KS} = \Sigma \lambda_i^+ / \ln 2$ bits per second. Equivalent readings: the rate at which microscopic uncertainty about the initial state is amplified into macroscopic uncertainty (information *generation*, Shaw 1981), or the rate at which predictive information about the future is lost (Engelken et al. quote bits per spike per neuron for spiking networks). One number, two framings.

6.2 All exponents negative: erasure and fading memory

$h\_KS = 0$. The sum of the exponents is the mean phase-space contraction rate; every exponent contributes to erasing where the system started, and the slowest one, λ_1, sets how long any of it survives: a perturbation decays as $e^{\lambda\_1 t}$, so the fading-memory timescale is $1/|\lambda\_1|$ (about 8 s in the multiple-timescale regime). The Kaplan-Yorke dimension is zero. The memory-capacity analysis measures the same thing from the decoding side.

6.3 Noise: recoverable information is SNR-limited

With noise, the question becomes what can be *decoded* from the state, and the answer is a Gaussian-channel bound. Information about an input of size δ injected at time 0 that remains recoverable at time t is approximately

$$I(t) \approx \tfrac{1}{2} \log_2 \left(1 + \delta^2 e^{2\lambda\_1 t} / \sigma\_x^2(t) \right)$$

per relevant direction, where $\sigma\_x^2(t)$ is the accumulated noise-driven variance along that direction.

- **Stable, $\lambda\_1 < 0$.** The signal shrinks while the noise sits at its stationary level; SNR decays as $e^{-2|\lambda\_1| t}$; recoverable information goes to zero exponentially. The input has not been destroyed by the dynamics; it has sunk below the noise floor. The crossing time $\ln(\delta/\sigma\_x)/|\lambda\_1|$ is the memory horizon the noise imposes.
- **Chaotic, $\lambda\_1 > 0$.** Signal and every noise kick injected since are amplified along the same unstable direction; $\sigma\_x^2(t) \sim \sigma^2 (e^{2\lambda\_1 t} - 1)/(2\lambda\_1)$, so the SNR tends to a constant $2\lambda\_1 \delta^2/\sigma^2$ and recoverable information about a past input *plateaus* rather than fading or growing. h_KS meanwhile keeps generating unpredictability, but what it generates is information about the noise history, not about the input.

This is the same fact that lets the noise cancel in Benettin's estimate, and it is why the reservoir results should be read as SNR-limited: the linear memory capacity is bounded by N in the noise-free case and noise lowers that bound in exactly this way. With a top-K spectrum, h_KS and D_KY come from the dynamics side and the memory-capacity curve estimates the SNR-limited recoverable information at each delay from the decoding side; the two describe one system.

7. Recommendations

1. **Implement top-K.** *Done 2026-09-12:* `lya_method = 'topk'` with `lya_K`, shared core `src/model/lyapunov/lyapunov_top` verified in `scripts/tests/test_lyapunov_topk.m` (Liouville, nesting, vs 'qr', vs Benettin). Reports $\lambda\_1..\lambda\_K$, h_KS, D_KY (+ resolved flag), conditioning. K is still chosen by hand: grow it until the smallest exponent is clearly negative and D_KY resolves.
2. **Run it on the full network, noise on and off**, in all three regimes, over the sweeps' reps. That replaces the reduced-network cross-check with a direct measurement and closes the noisy-QR gap.
3. **Keep Benettin (K = 1, finite difference) as the sweep estimator.** It is verified, cheap, and noise-cancelling. Where a stable-regime λ_1 is quoted to better than 0.05, either lengthen the warm-up to ∼60 s or quote the top-K value.
4. **Report medians over reps** for λ_1 in the chaotic regimes; consider h_KS as the headline chaos statistic, since its finite-time scatter is smaller and it scales with N in a way λ_1 does not.
5. **Manuscript wording.** "Benettin's method with a shared noise path" for the LLE; "Pesin's identity over the top-K spectrum" for h_KS; "finite-time exponents over 10 s, median over reps" wherever a single number appears.

References

- Benettin, Galgani, Giorgilli & Strelcyn (1980), Meccanica 15, 9-20.
- Shimada & Nagashima (1979), Prog. Theor. Phys. 61, 1605.
- Skokos (2010), The Lyapunov characteristic exponents and their computation, Lect. Notes Phys. 790. https://arxiv.org/abs/0811.0882

- Geist, Parlitz & Lauterborn (1990), Prog. Theor. Phys. 83, 875.
- Christiansen & Rugh (1997), Nonlinearity 10, 1063.
- Dieci, Russell & Van Vleck (1997), SIAM J. Numer. Anal. 34, 402; Dieci & Van Vleck, Computation of a few Lyapunov exponents, Appl. Numer. Math.; Dieci & Van Vleck (2008), SIAM J. Numer. Anal. 46, 1166.
- Talay (1991), SIAM J. Numer. Anal. 28, 1141; Carbonell, Biscay & Jimenez (2010), Int. J. Numer. Anal. Model. B 1, 147.
- Arnold (1998), Random Dynamical Systems, Springer.
- Engelken, Wolf & Abbott (2023), Lyapunov spectra of chaotic recurrent neural networks, Phys. Rev. Research 5, 043044. https://arxiv.org/abs/2006.02427
- Ginelli et al. (2007), Phys. Rev. Lett. 99, 130601 (covariant vectors).
- Noethen (2021), Strong fast invertibility and Lyapunov exponents. https://arxiv.org/abs/2112.11388
- Pesin (1977), Russ. Math. Surv. 32, 55. Shaw (1981), Z. Naturforsch. 36a, 80 (information generation).
- Gaspard & Wang (1993), Phys. Rep. 235, 291 (ε-entropy of noisy systems).